VECTOR SPACE · DEPLOYMENT GUIDE

Launching Vector Space

Publishing the site on GitHub Pages and connecting www.vectorspace.pk to it.

Written for someone who has never used GitHub.

No terminal, no code. About 40 minutes, plus waiting for DNS.

Ali Hashir · vectorspacepk@gmail.com

August 2026

BEFORE ANYTHING ELSE

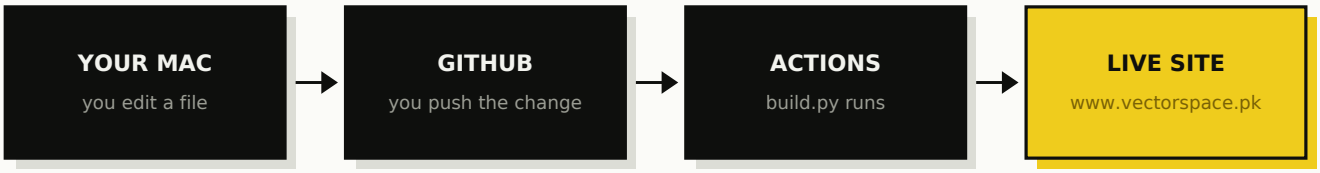
What you are actually doing

Right now the website exists only on your Mac. Publishing it means putting a copy on GitHub's computers, letting GitHub build it, and pointing your domain name at the result. That is the whole job — three moves.

There is one idea worth understanding first, because it explains why some of the steps look odd.

Your website is not stored as web pages. It is stored as ingredients. The `content/` folder holds plain text files, and `build.py` turns them into the 54-odd pages a browser can read. Those finished pages land in a folder called `docs/`.

You are not uploading *docs/*. You are uploading the ingredients and the recipe, and GitHub runs the recipe itself, on its own machine, every single time you push a change. That is what makes editing the site from your phone possible later.



Once this is set up, changing the site means editing a text file and clicking a button. Everything after that is automatic.

The route in front of you

	Step	What happens	Roughly
1	Set up GitHub	Free account, plus the GitHub Desktop app	10 min
2	Publish the folder	The first upload — about 282 MB	10–40 min
3	Switch Pages on	One dropdown, in repository settings	1 min
4	Watch it build	GitHub runs build.py and puts the site online	3 min
5	Test it	On a temporary github.io address, before the domain	5 min
6	Point the domain	Two kinds of DNS record at your registrar	10 min
7	Attach and secure	Tell GitHub the domain, then turn on HTTPS	5 min + wait
8	After launch	Search Console, sitemap, Instagram bio	15 min

Steps 6 and 7 involve waiting

DNS changes spread across the internet on their own schedule — usually 15 to 60 minutes, occasionally several hours. Nothing is broken during that window. If you want to launch on a particular day, do steps 1–5 the evening before.

PREPARATION

Before you start

Have these three things open and signed in. Gathering them now saves stopping halfway through with a half-published site.

1 — A GitHub account

Free, at **github.com**. Use an email you'll keep — build failure notices go there. Pick a username you don't mind being visible: it becomes part of your temporary web address, `yourname.github.io`. Something like `alihashir` or `vectorspacepk` is fine.

2 — GitHub Desktop

Download it from **desktop.github.com**. This is the app that moves files between your Mac and GitHub. It exists so that you never have to type a command. Install it, open it, and sign in with the account from step 1.

3 — Your domain registrar login

Wherever you bought **vectorspace.pk** — whoever takes your renewal payment. You need to reach the page that manages **DNS records** for the domain. It's usually labelled *DNS*, *DNS Management*, *Name Servers*, or *Advanced DNS*.

> **If your domain is at PKNIC**
>
> .pk domains are often managed through PKNIC directly, whose control panel is more basic than most and sometimes does not offer CNAME records at all. If you find you can only add A records, that is workable — there is a note in Step 6 explaining what to do instead. Check now rather than later.

What you will *not* need

- **A terminal or command line.** Not once. Everything happens in two apps and a browser.
- **Any payment.** GitHub Pages is free for public repositories. There is no hosting bill.
- **To understand Python.** The build script already works. GitHub runs it, not you.
- **A backup plan.** Your Mac keeps its copy of everything throughout. Nothing here deletes or moves your local files.

> **One decision to be aware of**
>
> GitHub Pages is only free on **public** repositories. Publishing this repo means the content files, the build script and all 282 MB of PDFs become publicly visible on github.com as well as on your site. Since every one of those files is already meant to be downloadable for free from the website, this changes nothing in practice — but it is worth knowing that it is a deliberate choice rather than an accident.

1 Set up GitHub

Create the account, install the app

Create the account

1. Go to **github.com** and click **Sign up**.
2. Enter your email, choose a password, choose a username.
3. Verify the email GitHub sends you.
4. When it asks about plans, choose the **Free** one. Skip anything about teams, Copilot or trials — none of it applies here.

Install GitHub Desktop

1. Go to **desktop.github.com** and download the Mac version.
2. Open the downloaded file and drag **GitHub Desktop** into Applications.
3. Launch it. On the welcome screen click **Sign in to GitHub.com**.
4. It opens your browser to authorise. Click **Authorize desktop**.
5. Back in the app it asks for a name and email for your commits. These get attached to every change you make. Your real name and email are fine.

> **What a 'commit' is**
>
> A commit is a saved snapshot of your files with a short note describing what changed — like a save point in a game, but with a label. The site's whole history is a chain of them, which is what makes it possible to look back at any earlier version. When GitHub Desktop asks for a 'summary' before letting you commit, it wants that label. Anything descriptive works: *Initial site*, *Fixed vectors worksheet*.

2 Publish the folder

The first upload — this is the slow one

In GitHub Desktop:

1. **File → Add Local Repository** from the menu bar.
2. Click **Choose…** and select the folder `vectorspace` — the one inside *Downloads / files 3*. Select the `vectorspace` folder itself, **not** *files 3* and not any folder inside it.
3. It will say the directory is not a Git repository, and offer to **create a repository**. Click that link.
4. A form appears. Leave **Name** as *vectorspace*. Leave Git Ignore and License as *None* — the folder already has its own ignore file. Click **Create Repository**.
5. You'll now see a long list of files in the left panel. That's everything about to be uploaded. At the bottom left, type **Initial site** in the Summary box and click **Commit to main**.
6. At the top, click **Publish repository**.
7. **Untick 'Keep this code private'.** This is the one setting that matters — Pages will not work on a free private repo. Then click **Publish repository**.

> **This upload is about 282 MB and will take a while**
>
> Almost all of it is past paper PDFs. On a normal home connection expect somewhere between ten and forty minutes. The progress bar sometimes appears to stall near the end while GitHub processes what it received — leave it be. This only happens once; every later change uploads in seconds because only the changed files travel.

Why the folder must be *vectorspace*, exactly

The build instruction GitHub follows says, in effect, *run build.py from the top of the repository*. If you accidentally publish *files 3* instead, then at the top of the repository GitHub finds a folder called `vectorspace` and no `build.py` — so the build fails immediately with *can't open file 'build.py'*. If you see that error later, this is why.

What actually gets uploaded

Uploaded	What it is
`build.py`	The script that turns your text files into web pages
`content/`	Every word on the site — settings, pages, topics
`static/`	PDFs, images, the stylesheet. This is the 282 MB
`templates/`	The page shell — header, footer, layout
`.github/`	The instructions GitHub follows to build and publish
`README.md, LAUNCH.md`	Your own notes. Harmless, and useful to keep

Not uploaded: `docs/` — the built site. It's deliberately excluded, because GitHub builds its own fresh copy. Also skipped: `.DS_Store` and other Mac clutter.

3 Switch Pages on

One dropdown. Get it right and everything works

Open your repository on **github.com** — GitHub Desktop has a **Repository → View on GitHub** menu item that takes you straight there.

1. Click **Settings** in the row of tabs across the top of the repository (not the settings under your profile picture — a different thing entirely).
2. In the left sidebar, click **Pages**.
3. Under **Build and deployment**, find the **Source** dropdown.
4. Change it to **GitHub Actions**.

Do not choose "Deploy from a branch"

It is the default, and it is wrong for this site. That option serves files straight out of the repository — but the finished pages aren't in the repository, they're built on demand. Choose it and you'll get an empty site or a 404, and it will look as though the whole thing is broken when the only problem is this dropdown.

Source must say: GitHub Actions.

You will not be asked to pick a workflow or paste anything in. The instruction file already sits in your repository at `.github/workflows/deploy.yml`, and GitHub finds it on its own. If it offers you a menu of starter workflows, close it — yours is already there.

There is nothing to save. The dropdown applies immediately.

What that file tells GitHub to do

Worth reading once, so the Actions tab isn't mysterious later. On every push to `main`, GitHub:

- starts a fresh Ubuntu machine,
- copies your repository onto it,
- installs Python 3.12,
- runs `python3 build.py` — which reads `content/` and writes `docs/`,
- takes the `docs/` folder and publishes it as your website.

The machine is then thrown away. Nothing persists between builds, which is why the site can never drift out of step with your content files — every build starts from nothing.

4 Watch the first build

Three minutes, and a green tick

Click the **Actions** tab at the top of your repository. You should see a run named **Build and deploy site**, with a yellow dot beside it meaning *in progress*.

Click into it to watch. There are two jobs — **build**, then **deploy** — and the second only starts when the first succeeds. The first run takes a few minutes, mostly spent moving PDFs around. Later runs are quicker.

What you see	What it means	What to do
Yellow dot	Still running	Wait. Refresh if impatient
Green tick	Built and published	Go to Step 5
Red X	Something failed	Read on
Nothing at all	No run was triggered	See below

If the Actions tab is empty, the push in Step 2 didn't complete, or Pages wasn't switched to GitHub Actions in Step 3. Check both, then use **Run workflow** on the *Build and deploy site* page to start one by hand.

If it fails

Click the red run, then click the failed step to expand its log. The useful line is near the bottom, and it is almost always one of these three.

The message says	What went wrong
`can't open file 'build.py'`	You published the wrong folder — see the note in Step 2. Fix by deleting the repository on GitHub and republishing the vectorspace folder itself.
`FileNotFoundError: content/...`	A content file didn't get uploaded, usually because it was still open and unsaved. Save it, commit, push again.
`Get Pages site failed / HttpError: Not Found`	Pages hasn't been switched to GitHub Actions yet. Do Step 3, then re-run the workflow.

A failed build cannot break your live site

If a build fails, GitHub simply doesn't replace what's already published — the previous version stays up. There is no state in which a mistake in a content file takes the site offline. Fix it whenever you get to it.

5 Test before you touch the domain

On the temporary address

Your site is now live at a temporary GitHub address, something like `https://yourname.github.io/vectorspace/`. The successful run in the Actions tab shows the exact URL on the **deploy** job.

> **Test here first, properly**
> Once the domain is attached, any problem you find becomes ambiguous — is it the site, or is it DNS? Ten minutes of checking now saves an hour of guessing later.

Work through this list

- The **homepage** loads and looks styled — Ink header, Bone background, yellow highlight behind *Free*. If it looks like unstyled black text on white, the stylesheet didn't publish and something is wrong with Step 3.
- **Further Mathematics** opens, and its paper dropdowns open and close. They should all start closed.
- A **topic page** — try Matrices — shows intro, syllabus points, common mistakes, then the resources.
- A **View PDF** button actually opens a PDF in a new tab.
- A **Coming soon** chip appears where a document isn't uploaded yet, rather than a broken link.
- **Additional Mathematics** works the same way.
- **A Level Mathematics** and **O Level Mathematics** are greyed out and not clickable, as intended.
- The **Courses** page shows the timetable table, and both buttons work — WhatsApp opens a chat, the other opens your Google Form.
- The **Found a mistake?** band appears on document pages and its email link works.
- Open the whole thing **on your phone**. This is the one most people skip, and most of your students will never see the site on anything else.

Don't worry about these yet

Links that point at **www.vectorspace.pk** won't work from the temporary address — the sitemap, the share preview, and a few absolute links are all built around your real domain. They come right the moment the domain is attached.

6 Point the domain at GitHub

Two kinds of DNS record

A domain name on its own does nothing; it needs a record saying which computer to go to. Those records live at your registrar — where you bought the domain — not at GitHub. Log in there, find **vectorspace.pk**, open its **DNS** page, and add two things.

A — The www record (the important one)

Your site is configured for **www.vectorspace.pk**, so this is the record that actually matters. One CNAME:

Type	Name / Host	Value / Points to	TTL
CNAME	www	yourname.github.io	Automatic, or 3600

Replace **yourname** with your GitHub username, in lowercase. If your registrar adds a trailing dot — `yourname.github.io.` — leave it; that's normal and correct. Note it is your *username*, not the repository: `vectorspace` does not appear anywhere in this record.

B — The bare domain records (so vectorspace.pk works too)

These catch people who type the address without *www*. Add all four — same name, four different values, which is expected rather than a mistake.

Type	Name / Host	Value
A	@	185.199.108.153
A	@	185.199.109.153
A	@	185.199.110.153
A	@	185.199.111.153

Registrars disagree about how to write the bare domain: `@`, an empty box, or `vectorspace.pk` in full. All three mean the same thing.

> **If your registrar won't let you add a CNAME on www**
>
> Some .pk control panels only offer A records. In that case, put the four A records above on **www** instead of on **@**, and it will work — you just lose GitHub's automatic failover if those addresses ever change. If the panel offers *ALIAS* or *ANAME*, use that on www with the value *yourname.github.io* — it behaves like a CNAME.

Delete anything that conflicts

If a record already sits on **www** or on **@** — often a parking page your registrar added when you bought the domain — delete it. Two records competing for one name give unpredictable results, and the parking page sometimes wins.

Then wait. Fifteen to sixty minutes is typical; a few hours is not alarming. Watch progress at **dnschecker.org** — enter *www.vectorspace.pk*, choose CNAME.

7 Attach the domain and secure it

The last configuration step

Back on github.com: **Settings → Pages**.

1. In the **Custom domain** box, type **www.vectorspace.pk** and click **Save**.
2. GitHub checks your DNS. If it complains that the record *could not be verified*, your DNS simply hasn't spread yet. Nothing is wrong. Leave it and come back in an hour.
3. Once verified, an **Enforce HTTPS** checkbox becomes available. Tick it.

HTTPS is the padlock in the address bar. GitHub issues the certificate free, but it takes a few minutes to a few hours after the domain verifies. During that window visitors may see a security warning. It resolves itself — don't go looking for a fix.

Your repository already contains a CNAME file — leave GitHub's box alone afterwards

When build.py runs, it writes a file called **CNAME** containing *www.vectorspace.pk*, taken from the *domain:* line in *content/site.txt*. That file is what keeps your custom domain attached through every rebuild.

The practical consequence: **if you ever change the domain, change it in content/site.txt**, not in GitHub's settings box. Change it only in the settings box and the next build will quietly overwrite you.

Confirming it worked

- **https://www.vectorspace.pk** loads your site, with a padlock.
- **vectorspace.pk** without the www redirects to the www version.
- **http://** redirects to **https://** automatically.
- The old *yourname.github.io* address now redirects to your domain too.

That's it. The site is live.

8 After launch

Worth an hour, not urgent

Tell Google the site exists

Google will find you eventually. Search Console makes it days instead of weeks, and shows you which topics students are searching for.

1. Go to **search.google.com/search-console** and sign in.
2. Add a property. Choose **URL prefix** and enter **https://www.vectorspace.pk**.
3. Verify — the HTML tag method is easiest, but it needs a line added to the site's head. Send me the tag and I'll add it.
4. Once verified, open **Sitemaps** in the sidebar and submit **sitemap.xml**. Your build generates it automatically, listing every page.

Check how it looks when shared

Send the link to yourself on WhatsApp. A preview card should appear with your image and the site name. This matters more than usual for you, because the link will mostly travel through WhatsApp and Instagram rather than search.

The preview image is currently a generic one I generated, at `static/img/og-default.png`. It works. Replace it whenever you want something better — 1200 × 630 pixels.

Then announce it

- Put **www.vectorspace.pk** in your Instagram bio.
- Post about the Further Pure 1 resources being live, since that's where the content actually is.
- Add the link to your WhatsApp profile.

Two small things still outstanding

- **The polar grid PDF** draws with 0.19pt and 0.4pt strokes. Browsers round those up to a full pixel, which is why it looks heavy on screen — it prints correctly. Say the word and I'll thicken them to about 0.35pt and 0.75pt so screen and print agree.
- **The privacy page** claimed you use analytics. You don't, so I corrected it before writing this. If you add analytics later, that page needs updating in the same commit.

FROM NOW ON

Changing the site once it's live

Every edit follows the same shape, whichever route you take: change a file, commit it, and about a minute later the live site has caught up.

Route A — from your Mac

1. Edit a file in the *content* folder in any text editor.
2. Open GitHub Desktop. Your change appears in the left panel.
3. Type a short summary — *Added vectors worksheet* — and click **Commit to main**.
4. Click **Push origin** at the top.

Best for: adding PDFs, or several edits at once.

Route B — from the browser, including your phone

1. Go to your repository on github.com and click into the file.
2. Click the pencil icon.
3. Edit, then click **Commit changes**.

Best for: fixing a typo when you're not at your desk. Works fully from a phone.

> **If you use both routes, pull before you edit**
>
> Edit in the browser, then later edit the same file on your Mac without pulling first, and you get two versions of the same file that Git can't reconcile — a merge conflict, which is tedious to untangle.
>
> The habit that prevents it entirely: **open GitHub Desktop and click Fetch origin (then Pull origin, if it offers) before you start editing.** Two seconds, every time. The same applies before you ask me to make changes — otherwise I'll be editing an old copy.

After every push, check the Actions tab shows a green tick. If a content file has a formatting mistake — a missing pipe character in a table row, say — the build stops and tells you which line. The live site keeps running the previous version until you fix it.

WHEN IT GOES WRONG

Troubleshooting

Symptom	Almost always	Fix
Site loads but has no styling — plain text on white	Pages source is set to a branch, not GitHub Actions	Settings → Pages → Source → GitHub Actions. Re-run the workflow.
404 on the github.io address	The build hasn't succeeded yet, or never ran	Actions tab. Look for a green tick. If there's no run at all, use Run workflow.
404 on www.vectorspace.pk but github.io works	DNS, or the custom domain isn't saved	Check the CNAME record points at yourname.github.io. Check Settings → Pages shows the domain.
Registrar's parking page appears instead of the site	An old record is still on www	Delete the conflicting record at your registrar, then wait an hour.
Certificate / 'not secure' warning	HTTPS certificate is still being issued	Wait. If it persists past a day, remove the custom domain, save, re-add it, save.
A change doesn't appear on the live site	Your browser is showing a cached page	Hard refresh: Cmd + Shift + R. The stylesheet is version-stamped, but pages aren't.
Build fails: can't open file 'build.py'	The wrong folder was published	The repository root must contain build.py. Delete the repo on GitHub and republish the vectorspace folder.
Build fails on a content file	A formatting slip — usually a missing | in a row	The log names the file and line. Fix, commit, push.
Push fails: 'updates were rejected'	You edited in the browser and on your Mac	Click Fetch origin, then Pull origin, then push again.
A PDF link gives 404	The file isn't in static/, or the name doesn't match	Filenames are case-sensitive on GitHub but not on your Mac — this is the usual culprit.

Nothing here is irreversible

The worst realistic outcome is a repository published wrongly, and the fix for that is to delete it on GitHub and publish again. Your Mac keeps the only copy that matters throughout. Deleting a GitHub repository never touches your local files.

PLAIN ENGLISH

The words you'll meet

Word	What it means here
Repository (repo)	The folder holding your site, plus its full history of changes. Yours is called vectorspace.
Commit	A saved snapshot with a note describing what changed. The history is a chain of these.
Push	Sending your commits from your Mac up to GitHub. This is what triggers publishing.
Pull / Fetch	The reverse — bringing down changes made elsewhere, such as an edit you made in the browser.
main	The name of the single line of history this project uses. You will never need another.
GitHub Actions	GitHub's system for running a task automatically when something happens. Yours builds and publishes the site on every push.
Workflow	One such task, written in a file. Yours is .github/workflows/deploy.yml.
GitHub Pages	The free web hosting. It serves whatever the workflow hands it.
DNS	The internet's address book — it turns a domain name into the address of an actual computer.
A record	A DNS entry pointing a name straight at a numeric address, like 185.199.108.153.
CNAME record	A DNS entry pointing one name at another name, rather than at a number.
TTL	How long other computers may cache a DNS answer before asking again. Leave it on automatic.
Propagation	The wait while a DNS change spreads. Usually under an hour.
HTTPS	The encrypted version of the web, and the padlock in the address bar. GitHub gives it to you free.
Static site	A site of pre-built pages with no database behind it. Fast, cheap, and almost impossible to hack — which is what yours is.

KEEP THIS PAGE

Reference card

The two settings people get wrong

Where	Must be
Settings → Pages → Source	GitHub Actions (not 'Deploy from a branch')
Repository visibility	Public (Pages is not free on private repos)

DNS records

Type	Host	Value
CNAME	www	yourname.github.io
A	@	185.199.108.153
A	@	185.199.109.153
A	@	185.199.110.153
A	@	185.199.111.153

Optional, for networks that use the newer addressing scheme — add these as well if your registrar supports AAAA records: 2606:50c0:8000::153, 2606:50c0:8001::153, 2606:50c0:8002::153, 2606:50c0:8003::153 — all on @.

Your addresses

Live site	https://www.vectorspace.pk
Repository	https://github.com/yourname/vectorspace
Builds	the Actions tab of that repository
Settings	Settings → Pages, in that repository
Sitemap	https://www.vectorspace.pk/sitemap.xml

Publishing a change

Edit a file in **content/** → GitHub Desktop → type a summary → **Commit to main** → **Push origin** → wait about a minute.

Pull first if you also edit in the browser.